

Legal AI Platform

Full Technical & Architecture Report

Backend Deep Dive + AI Stack + Frontend Overview + Diagrams

Generated from live codebase inspection

Workspace: C:\Users\ahmed\Desktop\pfe.2\legal-ai-platform

Report date: March 29, 2026

Scope: backend APIs, models, AI agents/services, architecture, and selected frontend coverage

System Inventory

Project footprint extracted from source code

- Backend routers scanned: 13
- Total API endpoints detected: 48 (DELETE:2, GET:22, POST:22, PUT:2)
- Top endpoint domains: intelligence (8), client_portal (7), rag (7), cases (5), clients (5), auth (3), document_router (3), voice (3)
- SQLAlchemy models: 11
- Pydantic API schemas: 63
- AI agent files: 10
- AI service modules: 21
- Frontend API client calls mapped: 18

Core Technology Stack

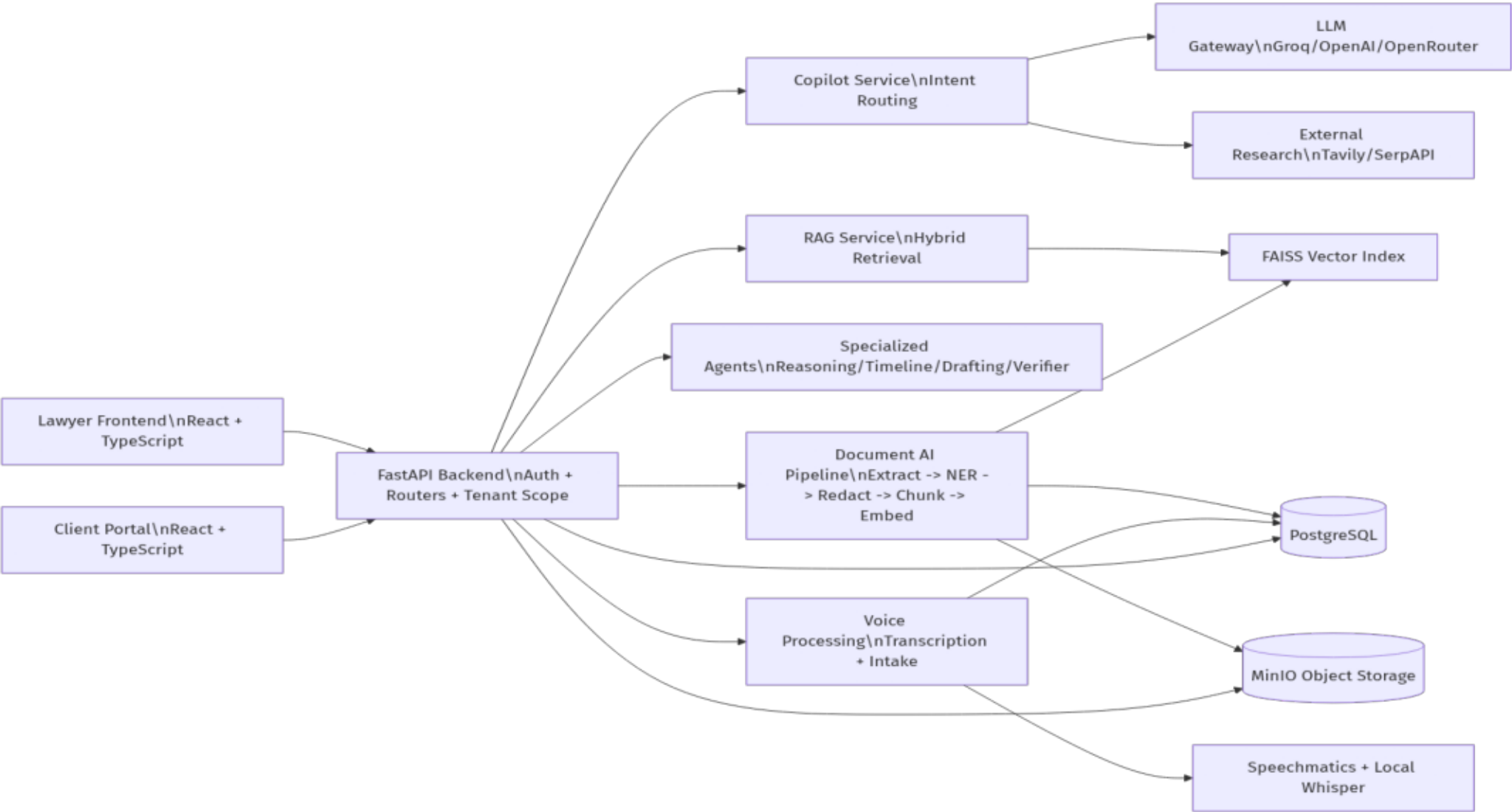
- Backend: FastAPI + SQLAlchemy + Pydantic Settings + JOSE JWT + Passlib
- Storage: PostgreSQL (primary), MinIO object storage, FAISS local vector index
- AI/NLP: OpenAI-compatible client gateway, Groq/OpenRouter/OpenAI compatibility, SentenceTransformers embeddings + reranker
- Transcription: Speechmatics API + local Whisper pipeline fallback
- Frontend: React + TypeScript single-page legal copilot desk + separate client portal app

Mermaid Diagram: System Architecture

Generated from architecture prompt + rendered from Mermaid source

Prompt sent to Mermaid: Create a professional system architecture flowchart for a Legal AI platform with two frontends (lawyer workspace and client portal), a FastAPI backend, a RAG service, a document AI pipeline, voice processing, and external integrations (LLM, FAISS, PostgreSQL, MinIO, etc).

Source: docs\mermaid\architecture_flow.mmd

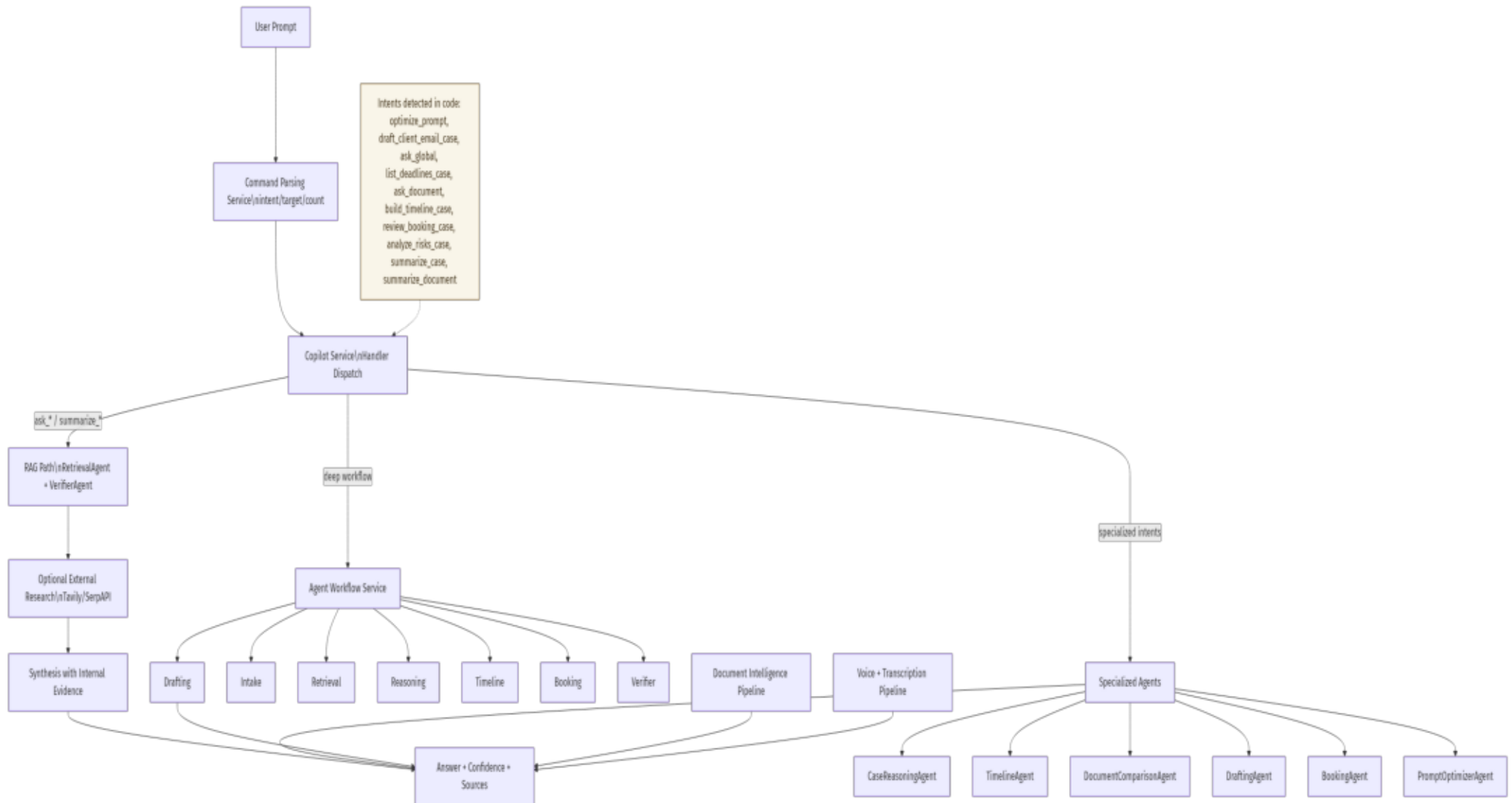


Mermaid Diagram: AI Orchestration

Intent routing, agent execution, grounding, and workflow paths

Prompt sent to Mermaid: Create an AI orchestration flowchart for a legal copilot pipeline: user prompt -> command parser -> intent routing -> either specialized agent

Source: docs\mermaid\ai_orchestration_flow.mmd

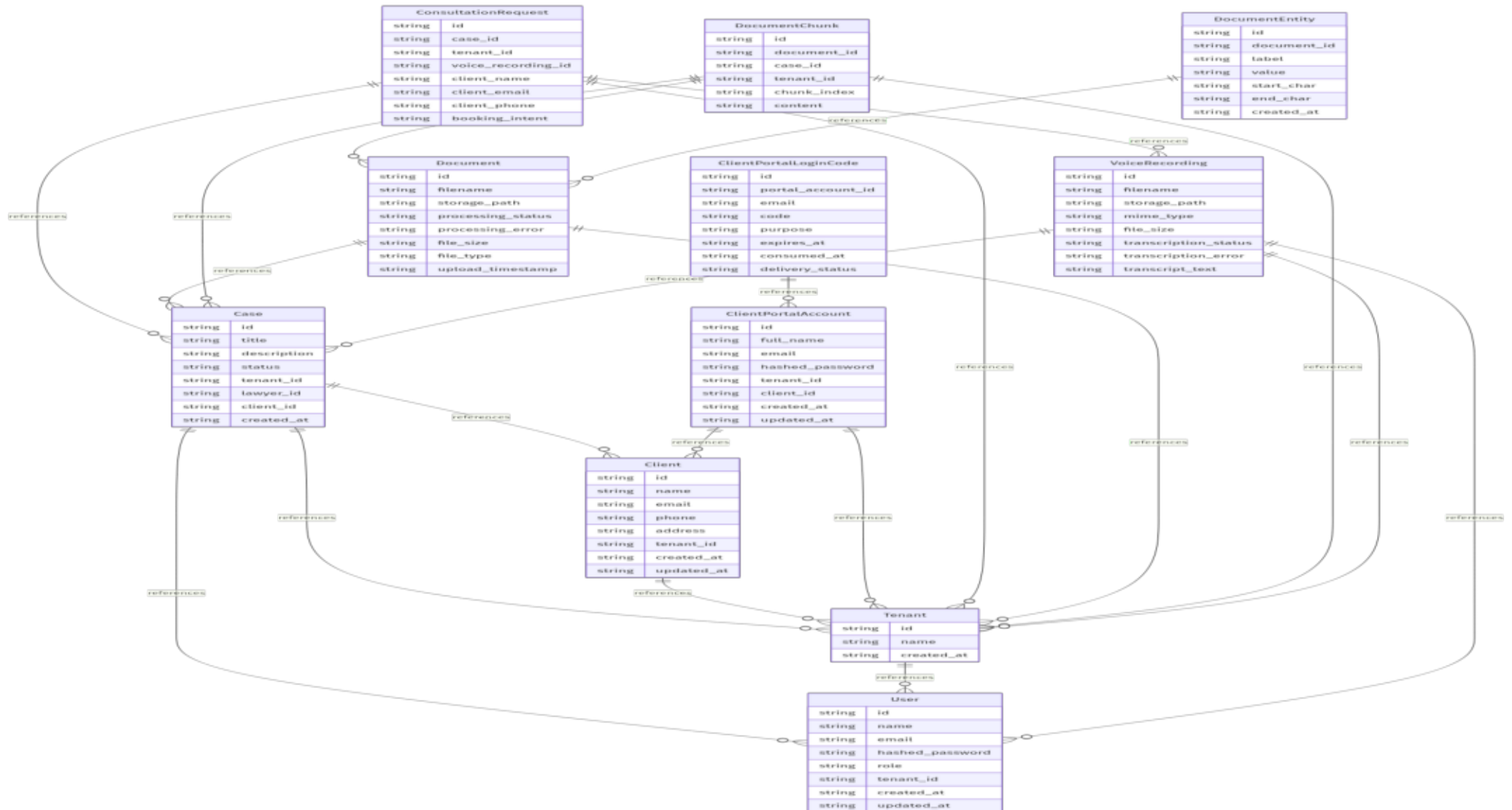


Mermaid Diagram: Data Model ER

Entity relationships derived from SQLAlchemy model inventory

Prompt sent to Mermaid: Create an ER-style diagram for the legal platform data model including tenant, users, clients, cases, documents, chunks, entities, voice record

Source: docs\mermaid\data_model_er.mmd



Backend AI Runtime

Capabilities implemented in services and agents

- LLM gateway supports OpenAI-compatible providers (current base URL: <https://api.groq.com/openai/v1>).
- Primary generation model: llama-3.3-70b-versatile; summary model: llama-3.3-70b-versatile.
- Copilot path combines intent parsing, targeted handlers, RAG, verifier checks, and optional external web research.
- Hybrid retrieval = lexical BM25 + semantic vector search + cross-encoder reranking.
- Document intelligence pipeline: extraction, cleaning, NER, PII redaction, chunking, embedding, indexing, summarization.
- Transcription supports Speechmatics first, then robust local fallback (openai/whisper-tiny).
- Reranker model: cross-encoder/ms-marco-MiniLM-L-6-v2 (default).

Detected AI service modules

- agent_workflow_service.py, chunking_service.py, command_parsing_service.py, copilot_service.py, document_ai_pipeline.py, document_classifier_service.py, document_insight_service.py, embedding_service.py, external_research_service.py, extraction_service.py, legal_text_formatter.py, llm_gateway.py, ner_service.py, pii_redaction_service.py, rag_service.py, reranker_service.py, summarization_service.py, text_cleaning_service.py, transcript_intake_service.py, transcription_service.py, vector_store.py

Detected copilot intents

- optimize_prompt, draft_client_email_case, ask_global, list_deadlines_case, ask_document, build_timeline_case, review_booking_case, analyze_risks_case, summarize_case, summarize_document, summarize_global, compare_case_documents, ask_case

Agent count: 10

Frontend Coverage

Main workspace UI + client portal

- Primary app is a one-page legal command center with sidebar, top runtime controls, copilot center, and contextual side panels.
- Integrated workspace tabs include workflow, intake, intelligence, and evidence surfaces.
- Dark mode is implemented through CSS custom properties and persisted in localStorage.
- Copilot composer supports scoped prompts, retrieval depth, external research toggle, and deep workflow trigger.
- Upload/record UX supports PDF evidence and voice-note ingestion with transcript display and intake generation.
- Client portal is separated from internal workspace and enforces client-safe feature boundaries.
- Client portal authentication includes password policy + email-based six-digit verification code flow.

Main app API usage (frontend/src/lib/api.ts)

- /auth/login, /auth/register, /auth/me, /cases/, /cases/, /clients/, /clients/, /documents/case/\${caseId}, /documents/upload?case_id=\${caseId}, /voice/case/\${caseId}, /voice/upload?case_id=\${caseId}, /consultations/case/caseId, /consultations/from – recording/{recordingId}, /intelligence/documents/\${documentId}/full-analysis, /ai/copilot, /ai/agent-workflow, /ai/provider-status, /ai/test-llm

Client portal API usage

- /portal/auth/login/request-code, /portal/auth/login/verify-code, /portal/auth/register, /portal/dashboard, /portal/intake/submit, /public/intake/\${reference}, /public/intake/submit

Security, Reliability & Operations

Cross-cutting controls implemented in code

- JWT authentication for staff and dedicated token type for client portal accounts.
- Rate limiter service protects staff login, portal login-code issuance, and code verification endpoints.
- Upload guardrails enforce content type and size limits for documents and voice inputs.
- Tenant-scoped queries are applied across API routes to isolate workspace data.
- Storage service sanitizes object names and ensures bucket readiness in MinIO.
- Request middleware injects request IDs and processing-time headers for traceability.
- Transcription path handles provider HTML/error anomalies and falls back to local inference.
- Schema patcher applies legacy DB compatibility updates on startup.

Provider key presence (.env, values redacted)

- GROQ_API_KEY: present
- OPENAI_API_KEY: not set
- TAVILY_API_KEY: present
- SERPAPI_API_KEY: not set
- SPEECHMATICS_API_KEY: present

Appendix A: Endpoint Catalog (1/2)

All detected API routes

```
[auth] POST    /auth/login
[auth] GET     /auth/me
[auth] POST    /auth/register
[cases] GET    /cases
[cases] POST   /cases
[cases] DELETE /cases/{case_id}
[cases] GET    /cases/{case_id}
[cases] PUT    /cases/{case_id}
[client_portal] POST    /portal/auth/login
[client_portal] POST    /portal/auth/login/request-code
[client_portal] POST    /portal/auth/login/verify-code
[client_portal] GET     /portal/auth/me
[client_portal] POST    /portal/auth/register
[client_portal] GET     /portal/dashboard
[client_portal] POST    /portal/intake/submit
[clients] GET    /clients
[clients] POST   /clients
[clients] DELETE /clients/{client_id}
[clients] GET    /clients/{client_id}
[clients] PUT    /clients/{client_id}
[consultations] GET    /consultations/case/{case_id}
[consultations] POST   /consultations/from-recording/{recording_id}
[document_router] GET   /documents/case/{case_id}
[document_router] POST  /documents/upload
[document_router] GET   /documents/{document_id}
[intelligence] GET      /intelligence/documents/{document_id}/entities
[intelligence] GET      /intelligence/documents/{document_id}/full-analysis
[intelligence] POST     /intelligence/documents/{document_id}/process
[intelligence] GET      /intelligence/documents/{document_id}/redacted
[intelligence] GET      /intelligence/documents/{document_id}/status
[intelligence] POST     /intelligence/documents/{document_id}/summarize
[intelligence] GET      /intelligence/documents/{document_id}/summary
[intelligence] POST     /intelligence/documents/{document_id}/summary/regenerate
[public] POST   /public/intake/submit
[public] GET    /public/intake/{public_reference}
```

Appendix A: Endpoint Catalog (2/2)

All detected API routes

```
[rag] POST    /ai/agent-workflow
[rag] POST    /ai/ask
[rag] POST    /ai/copilot
[rag] POST    /ai/process-document/{document_id}
[rag] GET     /ai/provider-status
[rag] POST    /ai/search
[rag] POST    /ai/test-llm
[search] GET  /search/documents
[sources] GET  /users
[users] GET   /users
[voice] GET   /voice/case/{case_id}
[voice] POST  /voice/upload
[voice] GET   /voice/{recording_id}
```

Appendix B: Backend Model Inventory

SQLAlchemy entities with key fields

Case (cases): id, title, description, status, tenant_id, lawyer_id, client_id, created_at

Client (clients): id, name, email, phone, address, tenant_id, created_at, updated_at

ClientPortalAccount (client_portal_accounts): id, full_name, email, hashed_password, tenant_id, client_id, created_at, updated_at

ClientPortalLoginCode (client_portal_login_codes): id, portal_account_id, email, code, purpose, expires_at, consumed_at, delivery_status

ConsultationRequest (consultation_requests): id, case_id, tenant_id, voice_recording_id, client_name, client_email, client_phone, booking_intent

Document (documents): id, filename, storage_path, processing_status, processing_error, file_size, file_type, upload_timestamp

DocumentChunk (document_chunks): id, document_id, case_id, tenant_id, chunk_index, content

DocumentEntity (document_entities): id, document_id, label, value, start_char, end_char, created_at

Tenant (tenants): id, name, created_at

User (users): id, name, email, hashed_password, role, tenant_id, created_at, updated_at

VoiceRecording (voice_recordings): id, filename, storage_path, mime_type, file_size, transcription_status, transcription_error, transcript_text

Appendix C: API Schema Classes (1/2)

Pydantic request/response contract classes

CaseCreate [case_schema.py]
CaseUpdate [case_schema.py]
CaseOut [case_schema.py]
ClientPortalRegisterRequest [client_portal_schema.py]
ClientPortalLoginRequest [client_portal_schema.py]
ClientPortalToken [client_portal_schema.py]
ClientPortalMessageResponse [client_portal_schema.py]
ClientPortalLoginCodeRequest [client_portal_schema.py]
ClientPortalLoginCodeVerifyRequest [client_portal_schema.py]
ClientPortalAccountOut [client_portal_schema.py]
ClientPortalConsultationItem [client_portal_schema.py]
ClientPortalDashboardResponse [client_portal_schema.py]
ClientCreate [client_schema.py]
ClientUpdate [client_schema.py]
ClientOut [client_schema.py]
ConsultationRequestOut [consultation_schema.py]
ConsultationFromTranscriptResponse [consultation_schema.py]
DocumentChunkOut [document_chunk_schema.py]
DocumentOut [document_schema.py]
DocumentListItemOut [document_schema.py]
DocumentAIProcessingOut [document_schema.py]
DocumentUploadResponse [document_schema.py]
AskRequest [intelligence_schema.py]
SearchRequest [intelligence_schema.py]
CopilotRequest [intelligence_schema.py]
RetrievedChunk [intelligence_schema.py]
RetrievalSearchResponse [intelligence_schema.py]
SourceItem [intelligence_schema.py]
AskResponse [intelligence_schema.py]
CopilotResponse [intelligence_schema.py]
EntityOut [intelligence_schema.py]
ProcessDocumentResponse [intelligence_schema.py]
ImportantDateItem [intelligence_schema.py]
DocumentSummaryOut [intelligence_schema.py]
StructuredDocumentInsightsOut [intelligence_schema.py]
DocumentSummarizeResponse [intelligence_schema.py]
FullDocumentAnalysisOut [intelligence_schema.py]
SearchResultItem [intelligence_schema.py]
IntelligenceSearchResponse [intelligence_schema.py]
PublicIntakeSubmitResponse [public_schema.py]
PublicIntakeStatusResponse [public_schema.py]
AskRequest [rag_schema.py]
SearchRequest [rag_schema.py]
CopilotRequest [rag_schema.py]
AgentWorkflowRequest [rag_schema.py]

Appendix C: API Schema Classes (2/2)

Pydantic request/response contract classes

LLMTestRequest [rag_schema.py]

RetrievedChunk [rag_schema.py]

SearchResponse [rag_schema.py]

SourceItem [rag_schema.py]

AskResponse [rag_schema.py]

CopilotResponse [rag_schema.py]

WorkflowStage [rag_schema.py]

AgentWorkflowResponse [rag_schema.py]

ProviderStatusResponse [rag_schema.py]

LLMTestResponse [rag_schema.py]

TenantCreate [tenant_schema.py]

TenantOut [tenant_schema.py]

UserRegister [user_schema.py]

UserLogin [user_schema.py]

UserOut [user_schema.py]

Token [user_schema.py]

VoiceRecordingOut [voice_schema.py]

VoiceUploadResponse [voice_schema.py]

Appendix D: AI Agent Catalog

Detected agent classes and public methods

BookingAgent (booking_agent.py): analyze_consultations

CaseReasoningAgent (case_reasoning_agent.py): analyze_case

DocumentComparisonAgent (document_comparison_agent.py): compare_case_documents

DraftingAgent (drafting_agent.py): draft_client_update_email

IntakeAgent (intake_agent.py): process_transcript

PromptOptimizerAgent (prompt_optimizer_agent.py): optimize_query

RetrievalAgent (retrieval_agent.py): retrieve

summarization_agent (summarization_agent.py): available, summarize_document

TimelineAgent (timeline_agent.py): build_case_timeline

VerifierAgent (verifier_agent.py): verify_answer